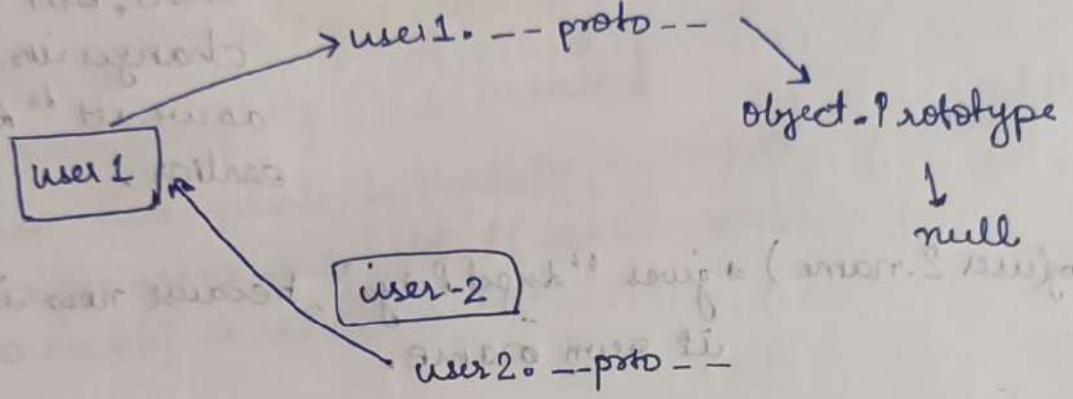
T-classes : Prototypes and Classes

What is Prototype?

Its a mechanism In which object inherits properties and method from other object.




```
const user1 = {
    name : "Suraj",
    score : "99",
    increment(){
        this.~~name~~ score + P
    }
}
const user2 = {
    --proto-- : user 1
}
```

console (user2.name) ⇒ Check if properties exist in user2, if not looks at second line --proto-- and now checks in the --proto-- of user 1, and finds name there, then prints it.

In the last code,

user2.name = "Angel Priya" ⇒ just adds it to user 2 but no changes in user 1.

2

user2.__proto__.name = "Dogesh" ⇒ This changes the name in user 1 also, but no changes in user2 as we set "Angel Priya" earlier.

console.log(user2.name) ⇒ gives "Angel Priya", because now it has it own name.

Note: If a object has its own property would reflect that property only, not inherit from another object.

eg: object2.__proto__.property ⇒ updates the reference of the object 1.

This is a old method, not recommended.

Can use Object.create

Now we can write

create a reference, not a copy

```
const user2 = Object.create(user1);
```

Constructor

It is a fn used to create multiple object with the same structure.

```
function User(name, score) {
    this.name = name;
    this.score = score;
}

const user1 = new User ("Jaani", 99)
```

So when you use new User();

1. A new empty obj {} is created
2. "this" here points to that empty object
3. New properties are added (name, score).
4. Now the obj is returned.

cf() [Constructor fun^n] → obj1 → new cf()

1. Create empty obj { }
2. Link → User.prototype
3. bind this → object
4. return obj

In this case all objects share methods from User.prototype

So,

user1.__proto__ = User.prototype ; gives true

Note: User.prototype is where methods are defined, and __proto__ belongs to the obj created using the cf, belongs to cf

log(user1 instance of User) gives true

Here "instance of" checks whether User.prototype exists in the obj's (User.prototype) if yes gives true.

Classes in JS

Classes is introduced to make the prototype thing easier to write and read.

eg:

```
class User {
    constructor(name, score) {
        this.name = name;
        this.score = score
    }
    increment() {
        this.score++
    }
}
```

```
const u1 = new User("Jaani", 100);
log(u1) => gives User { name : "Jaani",
                        score : 100
                      }
```

log(type of u1) gives object

log(type of User) gives function.

```
class User {
    constructor ( name , score) {
      this.name = name;
      this.score = score;
    }
    login () {
        log ("Logged In")
    }
}

class Admin extends User {
    constructor (name, score, isAdmin) {
      super (name, score)
      this.isAdmin = isAdmin;
    }
    deleteUser () {
    log ("deleted")
    }
}
const admin = new Admin ("Jaani", 100, true)
```

We used "super" keyword to access parent's class constructor and methods.

admin.login() ⇒ gives "logged In".

admin.deleteUser() ⇒ gives "deleted".

⇒ **Static**

```
class MathUtils {
    static add(a,b) {
        return a+b
    }
}
```

Here const m = new MathUtils()

m.add(2,5) ⇒ won't work bcs static fun belong only to class

So to access static fun,
use.
clog(MathUtils. add (2, 5)) : This works fine

Note: Static methods can be called on the object, but not on its objects.

Getter, setter

```
class User {
    constructor (name) {
      this._name = name
    }
    get name () {
        return this._name;
    }
    set name (value) {
        this._name = value;
    }
}

const u = new User ("Jaani");
```

log(u.name) ⇒ gives "Jaani" [Getter]

These are used to access and update the encapsulated data in a controlled way.

```
class Bank {
    #balance = 0;        ← private

    deposit (amount) {
        this.#balance += amount
    }
    balance () {
        return this.#balance
    }
const hdfc = new Bank();
log(hdfc.#balance) → can't access bcs private field
log(hdfc.balance()) → gives 0, bcs it is initialized
                      with 0
```

getter/setter lets us control how data is accessed and changed.

If getter/setter not used, people can update balance to '-1000' also to avoid that.

```
→ class Bank {
    #balance = 0;
    get balance() {
        return this.#balance
    }
    set balance (amount) {
        if (amount < 0) {
            log("Invalid amount")
            return;
        }
        this.#balance += amount
    }
```

Encapsulation : Is a concept of hiding internal data and controlling access to it through methods.

We have done that in last code using #private getter setter.

Abstraction :

```
class CoffeeMachine {
  #heat() {
  #brew() { }
  start() {
      this.#heat();
      this.#brew();
  }
}
```

Here in this code, we care about the start method, we don't how coffee is made internally. Here #heat() and #brew are private methods (hidden).

User just calls start() → coffee is ready.

Abstraction here, hiding private methods and exposing only start().

Polymorphism

```
class Animal {
    speak() {
      log("sound");
    }
}
```

```
class Dog extends Animal {
    speak() {
        log ("Bark");
    }
}
class Cat extends Animal {
    speak() {
        log ("Meow");
    }
}
```

} Same method behaves differently changing on object.

Polymorphism → same method, different behaviour

If you read my notes, do follow me :

x : jaaaani404

Happy learning!